Types of LLMs & Agents

Last amended: 6th Aug, 2026
My folder: C:\Users\ashok\OneDrive\Documents\llm
GitHub: https://github.com/harnalashok/LLMs/tree/main/install_ai_tools/misc
One reference is here and another one '*A comprehensive review of LLMs*' is here.

Table of Contents

Types of LLMs & Agents 1

Summary of LLM models 2

1. Foundation or Base models (`:text` models) 2
2. Chat models 2
3. Instruction Tuned models (`instruct` models) 3
4. Mixture of experts model (`moe` models) 4

What is MOE model: 4

Testing an MOE model 5

5. Reasoning models (`reasoning` models) 5
6. Embedding models 6

Made up examples 7

7. Multimodal models 9
8. Deep Research Agent 9

What are Deep Research Agents: 9

Example of a Deep Research Agent: 11

Use cases of multi-agent systems 11

9. Agent Orchestration 12

Sequential Agents 12

Hierarchical agents 13

Agent Networks 14

10. What is a Transformer model: 16

Example: 16

11. Decoder only models 17

Summary of LLM models

Exercise for students: Get list of example models from ollama library and HuggingFace

1. Foundation or Base models (`:text` models)

All generative LLMs are designed to take in some text and then predict what text is most likely to come after it.
"Base" models are trained on a wide variety of different texts, so they make minimal assumptions about the structure of the text they're completing. Training data on Internet is carefully filtered. For example, contents such as hate speech, adult content, fake news, personal-identifiable-information, information behind paywalls, dark web etc are all filtered out.
Foundation models in ollama are tagged as :text

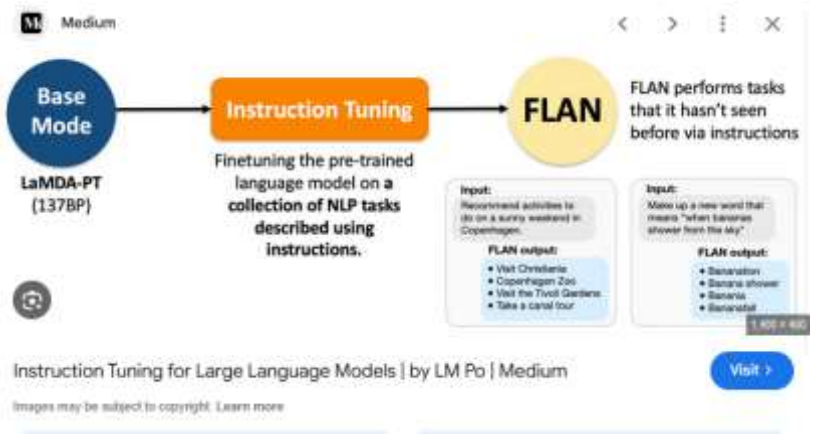
```
ollama pull llama2:text
ollama run llama2:text
Ask the question:  What is your name? ➔ It does NOT answer directly.
```

2. Chat models

Chat models are created from base models by training them on transcripts of high-quality dialogue, so they assume that whatever text they are given is a fragment of dialogue. You can chat with them by filling in one side of a conversation and letting the model fill in the other. (See this link for dataset on HuggingFace.)
Here are examples of chats from one of the training datasets:

input string · lengths 3↔50 3.7%	output string · lengths 1↔1.52k 100%
kya yaar,traffic mein stuck ho gaya.	arre, mere ko bhi late hoga ab!
kaunsa movie dekha tune?	bhai, wo new one, bollywood wala.
aaj ka khana kya banaya?	sabzi aur roti, usual stuff.
kal party hai na?	haan yaar, tu bhi aana.
koi plans hai weekend ke?	nahi yaar, just relaxing at home.
aapko kya problem hai?	kuch nahi, bas tension hai.

3. Instruction Tuned models (`instruct` models)



A typical dataset for instruction fine-tuning consists a) Instruction, b) Input Query and c) Output Response.

Instruction: "Translate the following English sentence to French."
Input: "The weather is nice today."
Output: "Le temps est agréable aujourd'hui."

Another example of instruction dataset:

Create a nested loop to print every combination of numbers between 0-9,…	Here is an example of a nested loop in Python to print every combination of numbers between 0-9,…
Write a function to find the number of distinct states in a given matrix. Each…	The given problem can be solved by iterating through each cell of the matrix and converting…
Write code that removes spaces and punctuation marks from a given string an…	Here's an example of code that attempts to solve the problem but contains an error related to…
Write a function that checks if a given number is prime or not. The function…	Here is an implementation of the function in Python: ```python import math def is_prime(n): …

One highly popular dataset for instruction fine tuning is here on HuggingFace. And here is a list of top 10% instruction tuning datasets on HuggingFace. Here is how to test an instruct model:

a. `ollama run granite4:350m`
b. `Ollama run llama3.2`

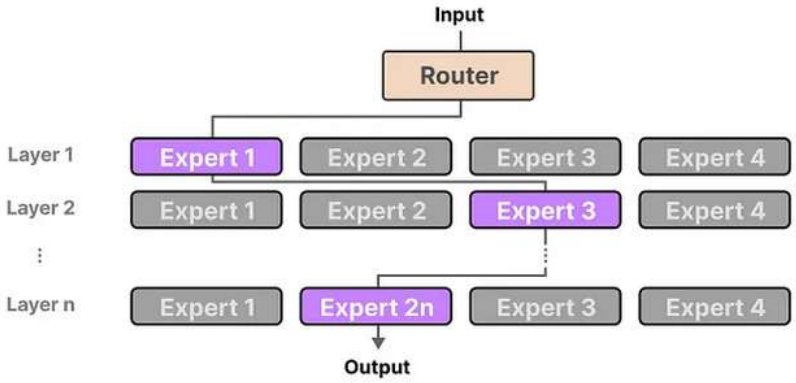
Give these instructions:

a. "Write an email in exactly 150 words." (Check with word count).
b. **Formatting constraints:** "Start every paragraph with an exclamation mark" or "Output the final answer in JSON."
c. **Forbidden elements:** "Write about coffee without using the letter 'e'."

Models often degrade when asked to perform multiple tasks simultaneously. Test composition by stacking rules to see where the model breaks:

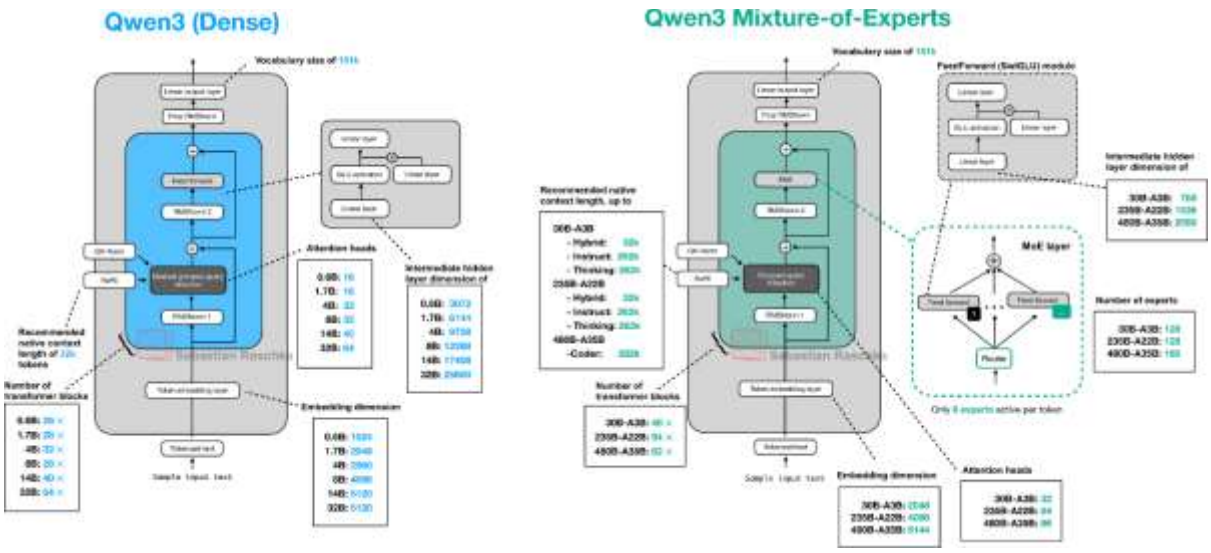
- *Level 1:* Summarize this article.
- *Level 2:* Summarize this article **in 3 sentences**, **without using the letter 't'**.
- *Level 3:* Summarize this article **in exactly 50 words**, **use bullet points**, and **cite at least two sources**.

4. Mixture of experts model (`moe` models)



What is MOE model:

An LLM is made of a pile of “layers”, each layer having both “attention heads” (which are responsible for understanding the relationship between words in the “context window”) and a “feed forward” block (a fully connected “multi-layer perceptron”, the most basic neural network), the later part is responsible for the LLM ability to store “knowledge” and represents the majority of the parameters.
MoE just comes from the realization that you don't need to activate the whole Feed forward block for every layer at all time, that you can split every feed forward blocks in multiple chunks (called the “experts”) and that you can have a small “router” in front of it to select one or several experts to be activated for each tokens instead of activating all of them. Thus, very simply, each expert is in fact, each Feed Forward layer.
The massively reduces the computation and memory bandwidth required to power the network while keeping its knowledge storage big.
Oh ,also what kind of knowledge is stored by each “expert” is unknown and there's no reason to believe that they are actually specialized for one particular task in a way that a human expert is.
And here is a workflow for MOE as compared to foundation models.



Testing an MOE model

You can evaluate an MoE model's capabilities using these specific prompt strategies:

- **Boundary & Multi-Domain Prompts:** Combine two wildly different topics in one prompt (e.g., *"Explain quantum mechanics using cooking metaphors while outlining 15th-century French history"*). This tests the router network's ability to engage multiple distinct experts simultaneously.
- **Specialized Depth Testing:** Ask hyper-specific, multi-step questions in a single domain (e.g., *"Provide an edge-case troubleshooting guide for Kubernetes cluster networking"*). This evaluates the true depth and capability of the individual experts.
- **Inconsistency & Hallucination Checks:** Ask the model to verify false premises or contrast complex ideas (e.g., *"Compare the economic policies of [Leader X] and [Leader Y], highlighting contradictions"*). MoE models often struggle more with consistency if the router network misdirects sub-tasks.

5. Reasoning models (`reasoning` models)

Reasoning models are trained by transitioning from predicting words to learning a step-by-step thinking methodology. While standard Large Language Models (LLMs) rely on massive data imitation, reasoning models are built using **large-scale Reinforcement Learning (RL) on verifiable problems**.

The process begins with a foundation or instruction-tuned model. AI developers use **Supervised Fine-Tuning (SFT)** by feeding the model a highly curated dataset filled with complex coding, math, and logic problems. Crucially, these examples don't just contain the correct final answer—they explicitly outline **detailed, step-by-step thinking traces**. This teaches the model the basic syntax of "thinking" before answering.

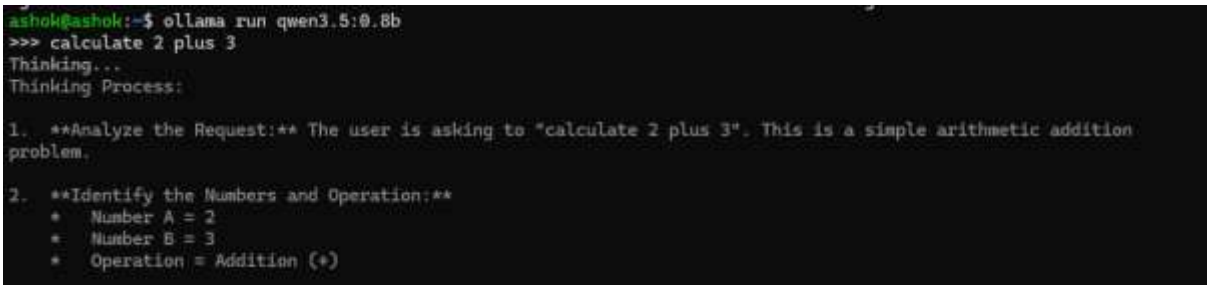


Figure 1: Thinking model output

Example Dataset Instance: Math Word Problem

1. The Prompt (The Input)

text

```
"A farmer has chickens and rabbits. There are 10 heads and 28 legs in total.
How many chickens and how many rabbits does the farmer have?"
```

Use code with caution.

2. The Target Response (The Gold Standard Output)

xml

```
<thought>
1. Identify the variables: Let C = number of chickens, R = number of rabbits.
2. Set up the first equation based on heads: C + R = 10.
3. Set up the second equation based on legs: Chickens have 2 legs, rabbits have
4. Try to solve by substitution. From equation 1, C = 10 - R.
5. Substitute C into equation 2: 2(10 - R) + 4R = 28.
6. Expand the equation: 20 - 2R + 4R = 28.
7. Simplify: 20 + 2R = 28.
8. Subtract 20 from both sides: 2R = 8.
9. Divide by 2: R = 4. So there are 4 rabbits.
10. Find chickens: C = 10 - 4 = 6.
11. Double-check the math:
    - Heads: 6 + 4 = 10. (Correct)
    - Legs: (6 * 2) + (4 * 4) = 12 + 16 = 28. (Correct)
12. Formulate the final user-facing response.
</thought>

The farmer has 6 chickens and 4 rabbits.
```

6. Embedding models

Execute the following line in ollama terminal to see vector for: `Financial bank`

```
Ollama run nomic-embed-text "Financial bank"
```

At their core, embedding models are designed to transform unstructured data into a high-dimensional, vector space. During transformation, each vector, or embedding, extracts or encapsulates the essential features of the input, preserving two things:

a) semantic relationships (as for example, between, *bear* and *fish; bears* and *fish* are semantically related, primarily through the thematic relationship of **predator and prey** in natural ecosystems. While they are not synonyms or hyponyms) and,

b) structural information (as for example '*bear ate the fish*' AND NOT *'fish ate the bear'* though they contain exactly the same four words).

These embeddings enable various downstream tasks, including semantic search, recommendation systems, clustering, classification, and more.

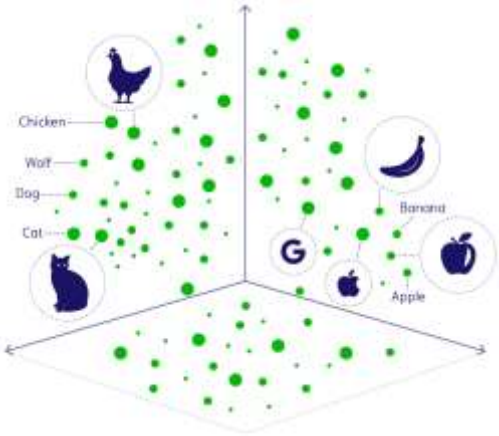


Figure 2: Similar objects fall in the same vector or coordinate space near to one another.

Embedding models are trained to ensure that similar inputs have embeddings that are close in the vector space, while dissimilar ones are placed far apart. This geometric arrangement in the embedding space allows for efficient similarity computations using distance metrics like cosine similarity or Euclidean distances.

Think of embeddings as **GPS coordinates for abstract concepts**. On a normal map, nearby coordinates mean two places are physically close. In an embedding space, nearby coordinates mean two concepts are *ideally* close.

Made up examples

Example1:

When data passes through an embedding model, it is converted into an array of numbers or **vector** containing hundreds or thousands of decimal numbers (dimensions). Each decimal number in the vector (also called dimension) tracks an abstract feature or relationship learned by the AI during training. Here is a made-up example of vectors for three words:

	[*Roy*, *Mas*,*Age*]	
Vector for King:	[0.99,0.95,0.7]	
Vector for Queen:	[0.99,0.05,0.7]	
Vector for Apple:	[0.00,0.15,0.7]	(0.15 being preferred by males; 0.7 for being Ripe apple as against green apple)

The numbers in each of the above three vectors may track *Royalty*, *Masculinity* and *Age* respectively. Thus:

Word	Dimension 1: Royalty	Dimension 2: Masculinity	Dimension 3: Age
King	High (0.99)	High (0.95)	Adult (0.7)
Queen	High (0.99)	Low (0.05)	Adult (0.7)
Apple	None (0.00)	Neutral (0.50)	N/A (0.0)

Figure 3: Each number in the array of numbers (vector) indicates some value assigned to some feature of the word.

Example2:

Here is another made up example on vector dimensions of *Violence*, *Entertainment value* and *Nationalistic sentiment*:

	Violence	**Entertainment value**	**Nationalistic sentiment**
Dhurandhar	0.9	0.7	0.9
3 Idiots	0.0	1.0	0.5
Dangal	0.1	0.8	0.95

Embedding model vectors may contain hundreds or thousands of decimal numbers (dimensions). The larger the vector space, more the semantic clarity and the structural relationships between words but would require more computational resources to process data. Smaller the vector space, less the computational power required but then less also would be semantic similarity.

Does *bank* in 'river bank' and *bank* in 'finance bank' have different vectors through embedding model?

Yes, the word "bank" will have different vectors in these context sentences if you use a modern context-aware embedding model. (Remember vectors store both semantic as also structural relationships).

Older embedding models like Word2Vec or GloVe give "bank" the exact same vector regardless of context. Modern Transformer-based models like BERT, GPT, or text-embedding-3 generate completely different vectors based on surrounding words

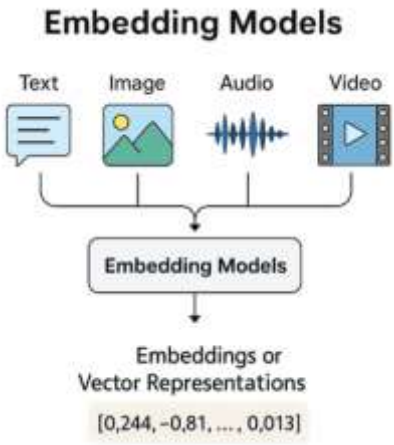


Figure 4: Vectors for 'apple', 'apple sound' and 'apple in picture' will be the same.

7. Multimodal models

Multimodal models are **artificial intelligence systems designed to process, understand, and reason across different types of data—or modalities—simultaneously**. Instead of only handling text like traditional chatbots, they can natively analyze combinations of **text, images, audio, video, and numerical data** to produce context-rich outputs. Google Cloud +3

How They Work

Unlike older systems that relied on separate AI tools (like one to "read" an image and another to describe it), multimodal models combine this processing into a single, unified architecture. Micron Technology +1

- **Encoding:** Specialized encoders translate different data types (e.g., pixel grids for images, audio waveforms) into a shared "language" of numerical vectors. IBM +1
- **Shared Vector Space:** Because different data types are converted into the same high-dimensional space, the model can inherently connect them (e.g., understanding that the spoken word "apple," a photo of an apple, and the written text "apple" all represent the same concept). YouTube · AssemblyAI +3
- **Unified Reasoning:** The AI uses its central engine to relate the inputs, infer hidden patterns, and generate responses—which can also be multimodal. IBM +1

Common Use Cases

The ability to combine data types unlocks dynamic real-world applications:

- **Visual Question Answering:** You can upload a photo of a broken appliance, ask the AI to identify the problem, and get text instructions on how to fix it. IBM +3
- **Data Interpretation:** The models can analyze charts, graphs, and financial reports simultaneously, outputting summarized insights. IBM +3
- **Audio/Speech Conversations:** You can talk directly to an AI assistant, and it will respond in natural, expressive spoken audio by understanding your tone, pacing, and words. IBM +1

MLLMs expand upon the foundations laid by traditional LLMs. While they often leverage a powerful pre-trained LLM as their backbone for language understanding and reasoning, they incorporate additional components to handle non-textual data. The core architectural difference lies in the need for specialized **encoders** for each modality. For instance, an MLLM might use a Vision Transformer (ViT) or a Convolutional Neural Network (CNN) to process images, an audio encoder for sound, and the standard LLM tokenizer/embedding layer for text. These encoders transform the input from each modality into vector representation. A crucial step in MLLM architecture is **embedding alignment and fusion**. The embeddings generated by the different modality encoders need to be projected into a shared space where the model can understand the relationships between them. A dedicated **fusion module** or specific training techniques (like contrastive learning) are employed to integrate these diverse representations into a unified multimodal vector space. This allows the model, for example, to connect the word “dog” in a text caption to the visual features of a dog in an accompanying image.

8. Deep Research Agent

What are Deep Research Agents:

Given a goal, an autonomous agent is one which can plan (say steps for execution), reason out (say how a step be executed), use correct tools among the set of tools and communicate its decisions. Ii is powered by an LLM model.

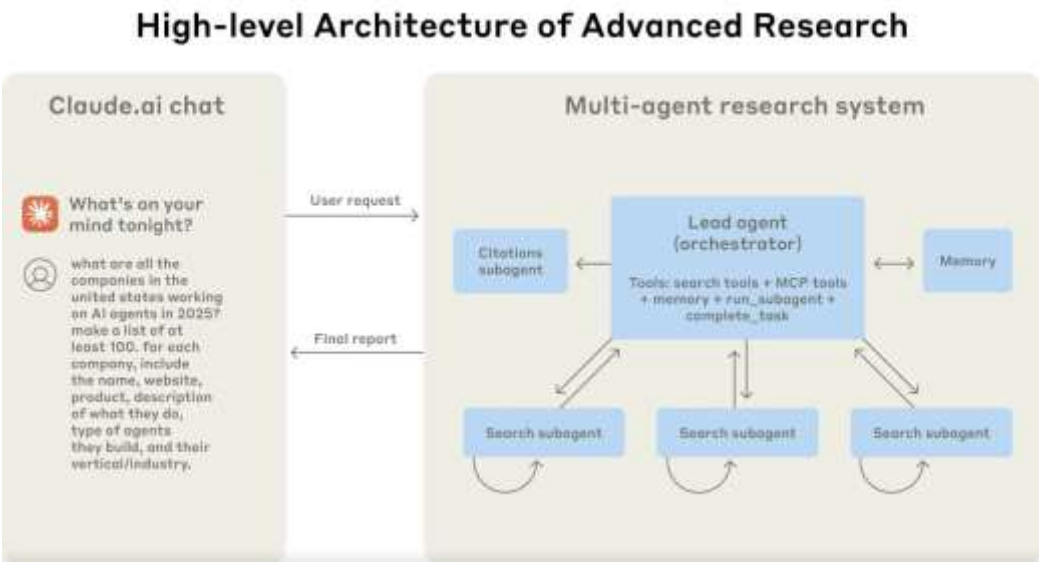
Deep research agents are **advanced AI systems designed to handle complex, open-ended research by planning autonomously, using tools iteratively, and maintaining extended memory**. Unlike standard chatbots that provide instant answers, these agents take minutes or hours to methodically browse the web, verify sources, and synthesize their findings into cited reports. Medium +1

Why Are They Different?

While traditional "shallow" AI agents often struggle when tasked with long, multi-step problems, deep research agents break through these limitations by using four core components: YouTube · LangChain +1

- **Autonomous Planning:** They generate dynamic to-do lists, break down broad questions into manageable sub-tasks, and adjust their strategy on the fly based on what they discover. YouTube · LangChain +1
- **Sub-agent Delegation:** Instead of relying on a single "jack-of-all-trades" model, the main agent can spawn specialized sub-agents (e.g., a dedicated researcher or a critique agent) to explore specific angles simultaneously. YouTube · LangChain +1
- **Persistent File Systems:** They use a virtual file space or long-term memory to collaborate, share context, and document partial findings without losing track over extended execution windows. LangChain +1
- **Evaluator Feedback Loops:** They continuously self-correct by having one AI model generate an answer while a secondary AI or an evaluator checks it for accuracy, restarting the search if the information is incomplete or incorrect. AnswerRocket

1. **Query Understanding and Planning:** Upon receiving a research query, the agent often starts by clarifying the scope and objectives with the user. It then formulates a plan, breaking down the research task into smaller, manageable steps.
2. **Information Gathering (Tool Use):** The agent autonomously uses tools, primarily web browsers, to search for relevant information online. It may access dozens or even hundreds of sources.
3. **Analysis and Synthesis:** The agent reads and analyzes the gathered information, extracting key points, identifying patterns, comparing data across sources, and potentially using other tools (like code interpreters for data analysis) to process the findings.
4. **Iterative Refinement:** The process is often iterative. Based on initial findings, the agent might refine its search queries, seek out additional sources, or revisit previous steps to deepen its understanding or resolve conflicting information.
5. **Report Generation:** Finally, the agent synthesizes its findings into a structured, often well-cited report, presenting the information in a coherent and organized manner.



Example of a Deep Research Agent:

Prompt: *How would I go about designing a Deep research Agent system that answers questions on diabetes. The system would consult reputed publications, especially the recent papers, published books, and other reputed resources on the subject on the Internet*.

Here's how to design a **Deep Research Agent system for Diabetes** using CrewAI:

Recommended Architecture

Use a **Hierarchical Process** with a manager agent coordinating specialized sub-agents:

Agents

1. **Manager Agent**— orchestrates the crew, delegates tasks, validates outputs
2. **Literature Search Agent**— finds recent academic papers (arXiv, PubMed-style searches)
3. **Web Research Agent**— searches reputed web sources (journals, health organizations)
4. **Synthesis Agent**— compiles and summarizes findings with citations

Key Tools to Use

Tool	Use
ArxivPaperTool	Find recent academic papers without an API key.
TavilyResearchTool	Conduct multi-step web research and produce cited reports.
SerperDevTool	Search Google for reputable publications.
ScrapeWebsiteTool	Extract content from selected journal or book pages.
PDFSearchTool	Search downloaded PDFs using RAG.

Use cases of multi-agent systems

Multi-agent systems (MAS) can solve many complex, real-world tasks. In general, MAS will find applications wherever there are multiple sensors working towards same shared objective—car sensors interacting with traffic light(s), car sensors interacting with toll-payment system, Security systems having multiple sensors. Some examples of applicable domains include:

Transportation

(Refer: this blog)

MAS can be used to manage transportation systems. The qualities of MAS that allow for the coordination of complex transportation systems are communication, collaboration, planning and real-time information access. Examples of distributed systems that might benefit from MAS are railroad systems, truck assignments and marine vessels visiting the same ports. Traffic Flow Agents monitor and analyse the ebb and flow of vehicles across city streets using Machine Learning. Using real-time data and predictive algorithms, these agents can:

- Detect congestion hotspots before they become gridlock nightmares
- Suggest alternative routes to drivers, helping to balance traffic loads
- Adjust traffic signal timings on the fly to optimize vehicle throughput

Healthcare and public health

Reference this blog

Multi-agent systems can be used for various specific tasks in the healthcare field. These agent-based systems can aid in disease prediction and prevention through genetic analysis. Wearable and IoT-connected agents monitor patient vitals in real-time. If anomalies (e.g., heart rate spikes or drops in oxygen) are detected, the system automatically alerts medical staff or triggers rapid response protocols. As patients undergo various tests, the MAS collect all structured, unstructured and image data and makes its analysis and recommendations on the proposed diagnostic method to be adopted. We may have Diagnostic Agent, *Risk-stratification* agent (may assess severity and predict potential outcomes. Morbidity and mortality risks are calculated using an established scoring system),

Supply chain management

Numerous factors affect a supply chain. These factors range from the creation of goods to the consumer purchase. MAS can use their vast informational resources, versatility and scalability to connect the components of supply chain management. To best navigate this intelligent automation, virtual agents should negotiate with one another. This negotiation is important for agents collaborating with other agents that have conflicting goals.[16]

Defence systems

Multi-agent systems can aid in strengthening defense systems. Potential threats can include both physical national security issues and cyberattacks. Multi-agent systems can use their tools to simulate potential attacks. One example is a maritime attack simulation. This scenario would involve agents working in teams to capture the interactions between encroaching terrorist boats and defense vessels.[17] Also, by working in cooperative teams, agents can monitor different areas of the network to detect incoming threats such as distributed denial of service (DDoS) flooding attacks.

9. Agent Orchestration

Reference this article on Medium and this IBM blog

Hierarchical multi-agent systems and agent networks (mesh or decentralized architectures) represent two distinct ways to organize artificial intelligence. A **hierarchical multi-agent system** acts like a corporate org chart with a clear chain of command (managers delegating to workers). An **agent network** functions like a decentralized web where peer agents collaborate and share information directly.

Sequential Agents

In CrewAI, sequential agents execute tasks linearly in a predefined order where the output of one task automatically serves as context for the next. This is the default execution strategy (Process.sequential) and is ideal for structured pipelines like a "Research-then-Write" workflow.

You can customize how data moves through your sequential pipeline using specialized Task parameters:

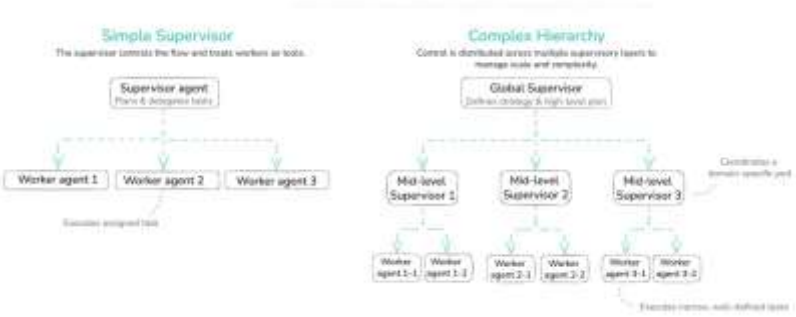
- **context**: If a downstream task needs data from multiple previous tasks (not just the immediate predecessor), you can explicitly pass them as an array: *context=[task_one, task_two]*.
- **async_execution=True**: If two tasks early in the pipeline do not rely on each other, you can run them in parallel. The sequential crew will wait for both to finish before passing their combined context to the next step.
- **output_json / output_pydantic**: Forces an agent to structure its handoff data into a rigid schema, preventing downstream parsing errors.

Hierarchical agents

Hierarchical Multi-Agent Systems

This architecture organizes agents into distinct tiers of authority and specialization, making it highly effective for breaking down complex, multi-step tasks.

- **Structure:** A top-level "supervisor" or "orchestrator" agent receives the primary user prompt, develops a strategic plan, and delegates focused subtasks to mid-level or worker agents.
- **Best For:** Long-horizon tasks that require strict quality control, structured planning, or heavy reasoning.
- **Advantages:**
  - **Context Management:** Higher-level agents only pass specific slices of context to workers, preventing context dilution.
  - **Tool Saturation:** Worker agents can be restricted to specific tools, reducing errors.
  - **Model Flexibility:** You can pair high-capacity frontier models for the "supervisor" with smaller, cheaper models for "workers".
- **Challenges:** Requires careful upfront design, and poorly executed handoffs can lead to the "telephone game" where errors propagate down the chain.



Real-World Examples of Hierarchical Agent Systems

Hierarchical structures excel in enterprise environments that require strict quality control, multi-step planning, and deep specialization.

- **Software Development Platforms (e.g., Devin, ChatDev):** A "Product Manager" agent takes the user prompt and writes specifications. It passes these to a "Chief Architect" agent to design the system. The architect then delegates coding tasks to multiple "Developer" agents, while a "QA" agent tests the code and reports bugs back up the chain.
- **Enterprise Financial Reporting:** A "Chief Financial Analyst" agent manages a quarterly report request. It delegates data retrieval to an "SQL Specialist" agent, data visualization to a "Python Analyst" agent, and compliance verification to a "Legal Auditor" agent before assembling the final executive summary.
- **Customer Support Escalation Hubs:** A "Triage Agent" analyzes an incoming ticket for sentiment and urgency. It routes complex technical issues to a "Tier 2 Technical Agent" and billing complaints to a "Billing Specialist Agent," retaining overall ownership of the customer ticket.

Agent Networks

Agent Networks (Decentralized/Mesh)

In a network or mesh architecture, there is usually no central coordinator. Instead, agents act as equals (peers) that talk directly to one another. Substack - Department of Product +1

- **Structure:** Agents form a peer-to-peer web where each node shares information with neighbors and negotiates tasks collectively to reach consensus. IBM +1
- **Best For:** Dynamic, rapidly changing environments where specialized agents need to negotiate or vote on the best course of action (e.g., autonomous vehicle fleets or logistics). Milvus +4
- **Advantages:**
  - **Resilience:** Because there is no single point of failure (like a supervisor), if one agent goes offline, the others can adapt.
  - **Flexibility:** Excellent for spontaneous collaboration.
- **Challenges:**
  - **Combinatorial Explosion:** In a full mesh network, connection loads can overwhelm the system. The number of potential connections scales quadratically with the number of agents, causing performance lag. GuruSup

Real-World Examples of Agent Networks (Mesh)

Network structures excel in decentralized, real-time environments where nodes must react dynamically to changing local conditions without waiting for centralized commands.

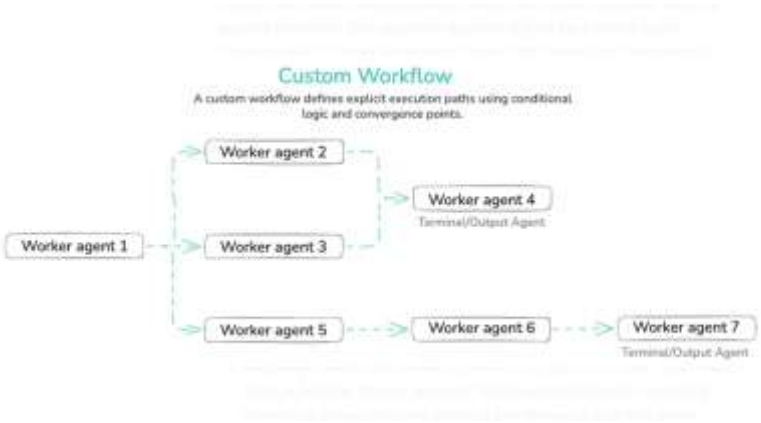
- **Autonomous Vehicle Fleets (Robotics):** Self-driving cars at an intersection form a localized mesh network. They directly negotiate speed, braking distance, and right-of-way with neighboring vehicles to optimize traffic flow and prevent collisions without a central server.
- **Decentralized Supply Chain Logistics:** Warehouse robots, delivery drones, and smart inventory shelves talk to each other as peers. If a drone is delayed by weather, it directly alerts the warehouse robot to prioritize a different package, adjusting the schedule through peer-to-peer negotiation.
- **Smart Grid Energy Management:** Individual houses with solar panels and batteries act as peer agents on an electric grid network. They buy and sell excess power directly to neighboring houses based on local supply and demand fluctuations, balancing the grid organically.
- **Multi-Model Voting Ensambles (MoE):** In advanced AI systems, multiple specialized peer models review a complex query simultaneously. They deliberate, vote, and cross-examine each other's reasoning to arrive at a consensus answer.

Core Comparison

Feature	Hierarchical Multi-Agent	Agent Networks (Mesh)
Control Flow	Top-down (tree structure)	Peer-to-peer (web structure)
Coordination	Managed by supervisors/orchestrators	Negotiated among equals
Scalability	High (easy to add new worker layers)	Low at scale (quadratic connection overhead)
Fault Tolerance	Lower (failure of a supervisor stalls the chain)	Higher (no single point of failure)
Optimal Use-Case	Complex, step-by-step reasoning (e.g., writing software, financial analysis)	Dynamic environments (e.g., robotics, gaming, IoT networks)

Custom Workflow

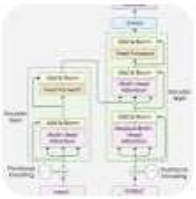
In a custom workflow architecture, you explicitly design the execution flow between agents. You decide how agents are connected and how tasks move through the system, using sequential steps, conditional branches, loops, or parallel execution. This approach does not rely on open-ended agent interactions or a central supervisor; instead, the behavior of the system is defined by the workflow itself, giving you full control over how it operates.



10. What is a Transformer model:

AI Overview

A **transformer model** is a revolutionary deep-learning neural network introduced in 2017 that powers modern generative AI. It processes sequential data (like words in a sentence) all at once rather than one by one, using an "attention" mechanism to understand how distant words relate to each other. NVIDIA Blog +1



How it Works

Unlike older models (like RNNs) that analyzed text in strict order, transformers rely on core building blocks to grasp meaning:

- **Self-Attention:** This is the magic of the transformer. The model calculates how much weight or "attention" it should assign to every word in a text sequence relative to every other word. This allows it to figure out context. For example, in the sentence "The bank of the river," self-attention helps the model understand that "bank" means the side of a river, not a financial institution, by linking the word to "river". NVIDIA Blog +3
- **Parallel Processing:** Because it doesn't need to read words sequentially, a transformer can process an entire book or dataset simultaneously, massively speeding up training and execution times. Coursera +1
- **Encoder-Decoder Architecture:** Traditional transformers are split into two parts. The **encoder** reads and compresses the input data, and the **decoder** translates it to generate the output. Coursera +1

Example:

Look at how the meaning of the exact same word changes completely across two different sentences:

Sentence A: *"The hunter sat on the muddy **bank** watching the ducks."*
Sentence B: *"The businessman walked into the brick **bank** to deposit cash."*

Without self-attention, a computer might look up "*bank*" in its digital dictionary and guess the wrong meaning. With self-attention, the word **"bank"** looks at its neighbours to define itself.
The model calculates an attention score between **"bank"** and every other word in Sentence A:

"bank" + "hunter" ➔ High connection (hunters sit outdoors).
"bank" + "muddy" ➔ Very high connection (nature/earth concept).
"bank" + "ducks" ➔ Very high connection (animals found near water).

The model has earlier read billions of sentences during training phase. And from that experience the model has adjusted its internal settings (called weight matrices). If the model is still under training, it will marginally further adjust its weight matrices. Once the model is finished training, its past-experience is completely frozen. When you hand it the new sentence—*"The businessman walked into the brick bank..."*—the model applies its frozen blueprints to the active words to understand what exactly '*bank*' means.

Another example:
The animal didn't cross the street because ***it*** was too tired.-➔ ***'it'*** is related to animal
The animal didn't cross the street because **it** *was too wide.--> **'it'** is related to street*

11. Decoder only models

In Artificial Intelligence, a decoder is called *autoregressive model* because it loops back (automatically) to consume its own past outputs (i.e. it regresses) as inputs to predict the next piece of data. Most modern LLMs are decoder-only models. Meta's Llama 3.2, for example, or OpenAI's GPT line of models—from the lightweight 1B and 3B versions to the massive 90B version—relies on a decoder-only pre-trained transformer architecture. The vast majority of household AI names operate on a decoder-only architecture. Imagine you ask an AI decoder (like a standard Large Language Model) to complete the phrase: **"The cat sat on the...".** The decoder runs a loop where its previous output becomes its next input:

Loop Step	Current Input String	Model Prediction (Next Word)
Step 1	"The cat sat on the"	**"mat"**
Step 2	"The cat sat on the **mat**"	**"and"**
Step 3	"The cat sat on the mat **and**"	**"fell"**
Step 4	"The cat sat on the mat and **fell**"	**"asleep"**
Step 5	"The cat sat on the mat and fell **asleep**"	`<END>` (Stop signal)

Every single time the model wants to type a new word, it has to read everything it has written up to that point. Because it feeds its own predictions back into itself, it is **autoregressive**. IBM +2

##################